

<<코드 리뷰>>

```
#include <stdio.h>
#include <sys/socket.h>
#include <unistd.h>
#include <stdlib.h>
#include <netinet/in.h>
#include <string.h>
#include <signal.h>
#include <errno.h>
#include <sys/wait.h>
```

- `stdio.h`: 표준 입출력 함수 사용
- `sys/socket.h`: 소켓 프로그래밍에 필요한 자료구조와 함수 선언
- `unistd.h`: UNIX 표준 시스템 호출 사용
- `stdlib.h`: 표준 라이브러리 함수 사용
- `netinet/in.h`: 인터넷 주소 구조체 정의
- `string.h`: 문자열 처리 함수 사용
- `signal.h`: 시그널 처리 함수 사용
- `errno.h`: 오류 번호 처리
- `sys/wait.h`: 자식 프로세스의 상태를 기다리는 함수 사용

```
void handle_client(int new_socket) {
    long valread;
    char *hello = "HTTP/1.1 200 OK\nContent-Type: text/plain"
                  "Content-Length: 20\n\nMy first web server!";
    char buffer[30000] = {0};

    valread = read(new_socket, buffer, 30000);
    printf("%s\n", buffer);

    write(new_socket, hello, strlen(hello));
    printf("-----Hello message sent-----\n");

    close(new_socket);
}
```

- 클라이언트로부터 데이터를 읽고, 간단한 HTTP 응답을 작성하여 클라이언트에게 보냄
- 응답 메시지는 "HTTP/1.1 200 OK"와 간단한 텍스트 콘텐츠를 포함.

```
void sigchld_handler(int s) {
    int saved_errno = errno;
    while(waitpid(-1, NULL, WNOHANG) > 0);
    errno = saved_errno;
}
```

`waitpid` 함수를 사용하여 모든 종료된 자식 프로세스를 수거

```

int main(int argc, char const *argv[]) {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    if (sigaction(SIGCHLD, &sa, NULL) == -1) {
        perror("sigaction");
        exit(1);
    }
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("In socket");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    memset(address.sin_zero, '\0', sizeof address.sin_zero);
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("In bind");
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 10) < 0) {
        perror("In listen");
        exit(EXIT_FAILURE);
    }
    while (1) {
        printf("\n+++++++ Waiting for new connection ++++++\n\n");

        if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen)) < 0) {
            perror("In accept");
            exit(EXIT_FAILURE);
        }
        if (!fork()) {
            close(server_fd);
            handle_client(new_socket);
            exit(0);
        }
        close(new_socket);
    }
    return 0;
}

```

sigaction 함수를 사용하여 **SIGCHLD** 시그널을 처리하도록 설정하고 socket을 이용해 소켓을 생성함

bind는 소켓의 주소를 바인딩 하고 listen 함수를 사용하여 연결대기 상태로 만듦 ,accept()함수는 실질적으로 연결을 수락하는 함수임.

<<특이사항>>

: sleep()의 인자의 값을 적절하게 조절하면서 테스트를 수행했고, 특별히 코멘트 남길 만한 상황은 없었습니다.